You said:

Lab_4_VibeCoding_...
PDF

https://github.com/SETAPESU26/47_target-aim-trainer

walk me through this assignment

Worked for 8s

ChatGPT said:

You said:

yes i opened readme

ChatGPT said:

You said:

Target Aim Trainer

This project is a terminal-based target aim trainer using **Pygame**. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

What's Provided

A partially working version of an aim trainer with:

- A single target that spawns at a random position and visually shrinks the longer it's on screen
- Click-to-hit scoring, with a timed-out target counting as a miss
- A countdown timer, score, and accuracy display

You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Clone the repo or download the project folder.
2. Make sure you have Python 3.10+ installed.
3. Install dependencies:

```
pip install -r requirements.txt
```

4. Run the game:

```
python main.py
```

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

> Late clicks well outside the small, shrunken target circle can still register as a hit. Investigate and enhance click accuracy so the clickable area always matches what's actually drawn on screen.

Task 2: Implement Game Over Condition

> Add a screen that displays the final score and accuracy once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard target lifespan/size), or exit.

Task 4: Add Sound Feedback

Add basic sound effects for a successful hit, a miss (including a timed-out target), and the round ending.

Expected Behavior

- A single target appears at a random position and shrinks the longer it stays on screen
- Clicking the target scores a hit and immediately spawns a new one elsewhere
- Clicking anywhere else counts as a miss; letting a target's timer run out also counts as a miss
- A countdown timer, score, and live accuracy percentage are visible at all times
- The round ends when the timer reaches zero

Folder Structure

```
target-aim-trainer-main/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   └── target.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

ChatGPT said:

You said:

yes i forked it

ChatGPT said:

You said:



i was working on a previous project. what do i do

ChatGPT said:

You said:

yes the window opened

ChatGPT said:

You said:



which do i select

ChatGPT said:

You said:

ok done

ChatGPT said:

You said:

ok i opened in vs code

ChatGPT said:

You said:

yes the game opened

ChatGPT said:

You said:

ok i recorded it

ChatGPT said:

You said:

```
class Target:
    def __init__(self, x, y, base_radius=40, min_radius=12,
lifespan_frames=90):
        self.x = x
        self.y = y
        self.base_radius = base_radius
        self.min_radius = min_radius
        self.lifespan_frames = lifespan_frames
        self.age = 0
```

```
def update(self):
    self.age += 1

def expired(self):
    return self.age >= self.lifespan_frames

def visual_radius(self):
    # The target shrinks as it ages, giving the player less time
    # to react the longer it's been on screen.
    t = min(1.0, self.age / self.lifespan_frames)
    return self.base_radius - (self.base_radius - self.min_radius) * t

def contains_point(self, x, y):
    # NOTE: hit-testing always uses the target's starting radius
    # (base_radius), not its current, shrunken on-screen radius
    # (visual_radius()). The circle drawn in render() shrinks as
    # the target ages, but the clickable area never does, so a
    # late click well outside the small visible circle can still
    # register as a hit. See Task 1 in the README.
    return math.hypot(self.x - x, self.y - y) <= self.base_radius
```

ChatGPT said:

You said:

ok done

ChatGPT said:

You said:

yes it works fine

ChatGPT said:

You said:

ok next

ChatGPT said:

You said:

ok done

ChatGPT said:

You said:

```
import pygame
import random
from .target import Target

# Game Engine

WHITE = (255, 255, 255)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.margin = 60
        self.hud_height = 60
        self.target = self._spawn_target()

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds * 60

        self.hits = 0
        self.misses = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 26)
        self.game_over = False

    def _spawn_target(self):
        x = random.randint(self.margin, self.width - self.margin)
        y = random.randint(self.margin + self.hud_height, self.height -
self.margin)
```

```
        return Target(x, y)

    def handle_event(self, event):
        if self.game_over:
            return
        if event.type == pygame.MOUSEBUTTONDOWN:
            self._handle_click(event.pos)

    def _handle_click(self, pos):
        x, y = pos
        if self.target.contains_point(x, y):
            self.hits += 1
            self.score += 1
            self.target = self._spawn_target()
        else:
            self.misses += 1

    def handle_input(self):
        # Reserved for continuously-held-key input; this game is
        # entirely mouse-driven, so there's nothing to poll here.
        pass

    def update(self):
        if self.game_over:
            return

        self.time_left_frames -= 1
        if self.time_left_frames <= 0:
            self.game_over = True
            return

        self.target.update()
        if self.target.expired():
            self.misses += 1  # letting a target time out counts as a miss
too
            self.target = self._spawn_target()

    def accuracy(self):
        total = self.hits + self.misses
        if total == 0:
            return 0.0
        return round(100 * self.hits / total, 1)
```

```
    def render(self, screen):
        r = int(self.target.visual_radius())
        pygame.draw.circle(screen, RED, (self.target.x, self.target.y), r)
        pygame.draw.circle(screen, WHITE, (self.target.x, self.target.y), r,
2)

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))

        seconds_left = max(0, self.time_left_frames // 60)
        timer_text = self.font.render(f"Time: {seconds_left}s", True,
WHITE)
        screen.blit(timer_text, (self.width - 140, 10))

        acc_text = self.font.render(f"Accuracy: {self.accuracy()}%", True,
WHITE)
        screen.blit(acc_text, (self.width // 2 - 90, 10))

        if self.game_over and not getattr(self, "_game_over_logged",
False):
            # NOTE: no proper game-over screen yet - see Task 2 in the
README.
            print(f"Time's up! Final score: {self.score}  Accuracy:
{self.accuracy()}%")
            self._game_over_logged = True
```

ChatGPT said:

You said:

yes it works

ChatGPT said:

You said:

ok next

ChatGPT said:

You said:

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 500
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Target Aim Trainer - Pygame Version")

# Colors
DARK_GRAY = (35, 35, 40)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(DARK_GRAY)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
```

```
if __name__ == "__main__":
    main()
```

ChatGPT said:

You said:

yes it works

ChatGPT said:

You said:

ok next

ChatGPT said:

You said:

ok i added it

ChatGPT said:

You said:

yes it works

ChatGPT said:

You said:

how do i export the chat?

ChatGPT said: